Criterion B: Design

Preliminary explanation table for main variables and data structures

NAME	TYPE	FUNCTIONALITY
User	Class	It contains all the functions attributed to Bill as a user (e.g create account, log-in etc).
BudgetProgram	Class	Contains all functions related to budget entries (e.g category, amount, dates to pay, sorting, financial projections)
BankProgram	Class	Used to handle banking operations (e.g deposit and withdrawals)
catfordeletion	List	Contains every row with budgetary data that is to be removed
dates	List	List containing all the dates to be paid of each budgetary data/row entered by Bill. Will be primarily used for calculating when his

		money will run out based on average spending
amounts	List	List containing all the amounts to be paid of each budgetary data/row entered by Bill. Will be primarily used for calculating when his money will run out based on average spending
storetuples	List	Stores each row of budgetary data in a list for sorting

Explanation table for main functions

NAME	FUNCTIONALITY
createacc()	Function to allow Bill to create his own account based on specific parameters set (e.g upper and lower case letters for password). Will also allow for password obfuscation through hashing and salting.
uploadtodrive()	Function to upload database to a

	Google Drive folder
.execute()	Mostly used for executing operations regarding the database (e.g creation of tables)
Frame()	Initializes a new tab regarding the GUI
sortcolumn()	Used to reverse values regarding each column for budgetary data based on Bill's preference (by clicking the name of the column)
loginacc()	Used to check whether the information for login entered by the country satisfies the information within the database when Bill's account was first created. Furthermore, it should be able to unhash the password in the database to allow Bill to enter the application.
showbank()	Shows all GUI functionalities related to the banking program (e.g any buttons related to withdrawal, deposits)
showwith()	Shows all GUI functionalities related to the withdraw amount tab
withamount()	Allows Bill to withdraw a specific amount from his account balance with the click of the withdrawal button. If Bill attempts to withdraw more money than available in his account balance, enter a string, or a negative amount, an appropriate error message should be shown.

showdep()	Shows all GUI functionalities related to the withdraw amount tab
depamount()	Allows Bill to withdraw a specific amount from his account balance with the click of the deposit button. If Bill attempts to enter a string, or a negative amount, an appropriate error message should be shown.
showbudget()	Shows all GUI functionalities related to the budget tab. Treeview is to be used here to show Bill's budgetary data stored in the database
displaydata()	Fetches all of Bill's budgetary data within the database to be shown in Treeview as he logs in again to his account.
showadd()	Showcases every GUI element related to the tab for addition of new rows/budgetary data
validateemail(email)	Checks whether the format of the email entered by Bill follows the appropriate syntax of emails.
sendemail()	Used to send Bill's budget to his gmail on command.
removecat()	Allows for the selection of all rows/budgetary data to be removed from the database based on Bill's command
addcat()	Allows for the creation of rows/budgetary data to be added from the database based on Bill's

	command. Will allow for the input of dates to be paid, amounts to be paid, categories, and subcategories. Appropriate functions for input validation are to be created (e.g for checking the date to be paid format or if Bill actually entered everything in each entry)
projection()	Allows to check when Bill's money will run out based on average spending, dates to be paid, and his account balance. A financial projection using a graph created with Matplotlib will be provided based on Bill's command.
uploadtodrive()	Uploads the database constantly to a Google Drive folder for extra security

GUI Design:

The following features all the preliminary designs of the GUI for each section of the application. There were not any particular GUI specifications handed in by the client.

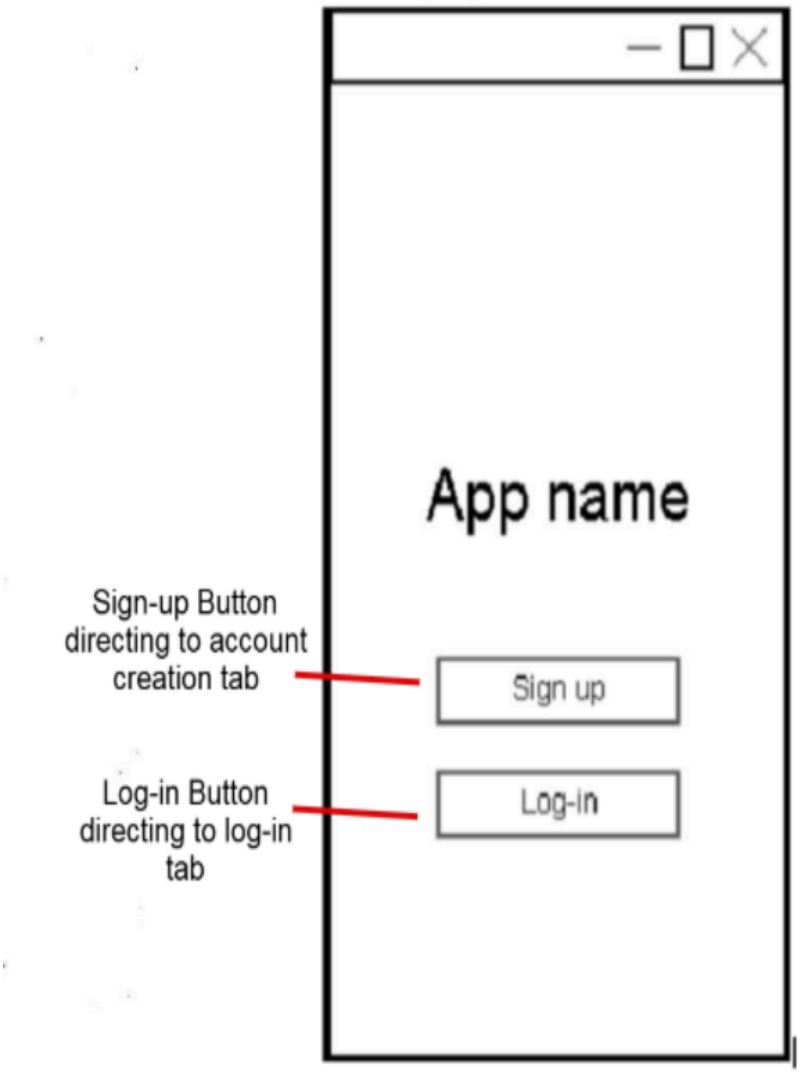


Figure 1: User account management page

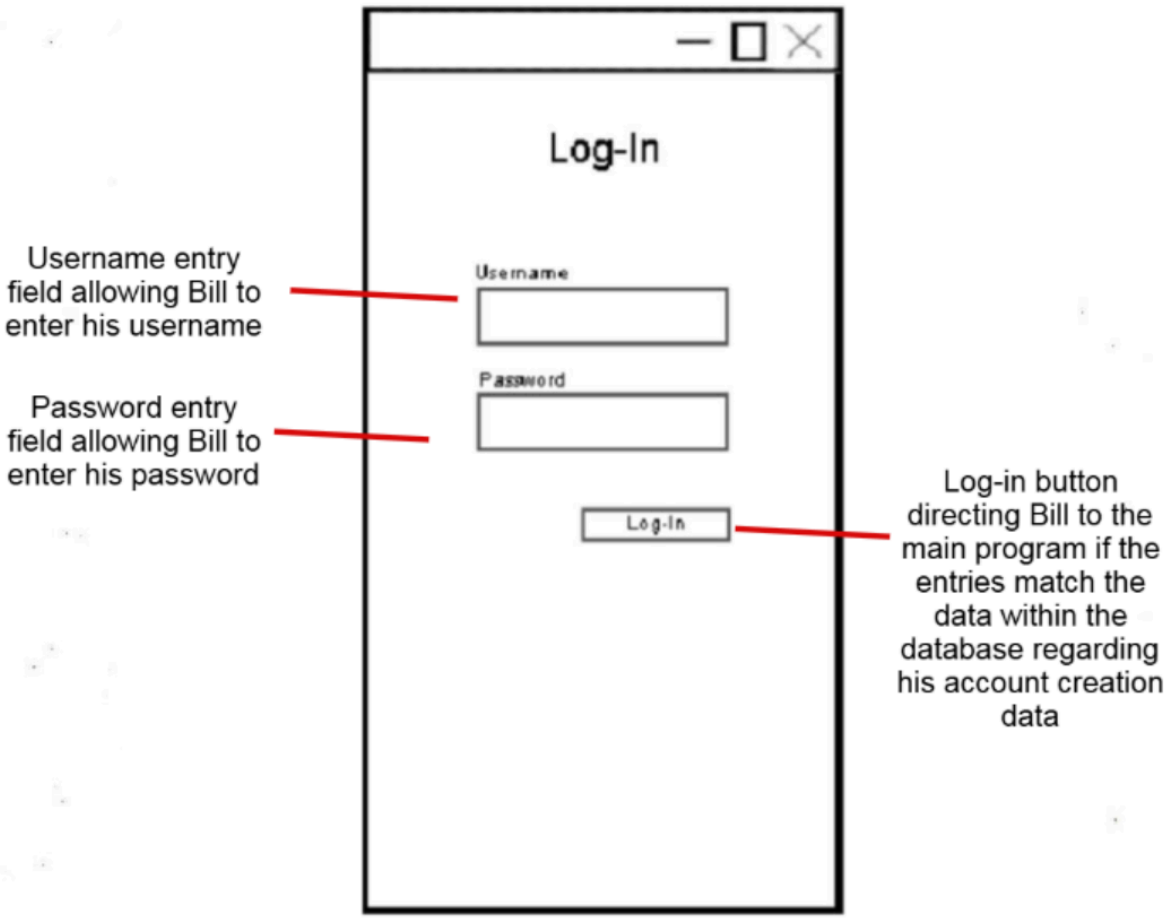


Figure 2: Log in tab

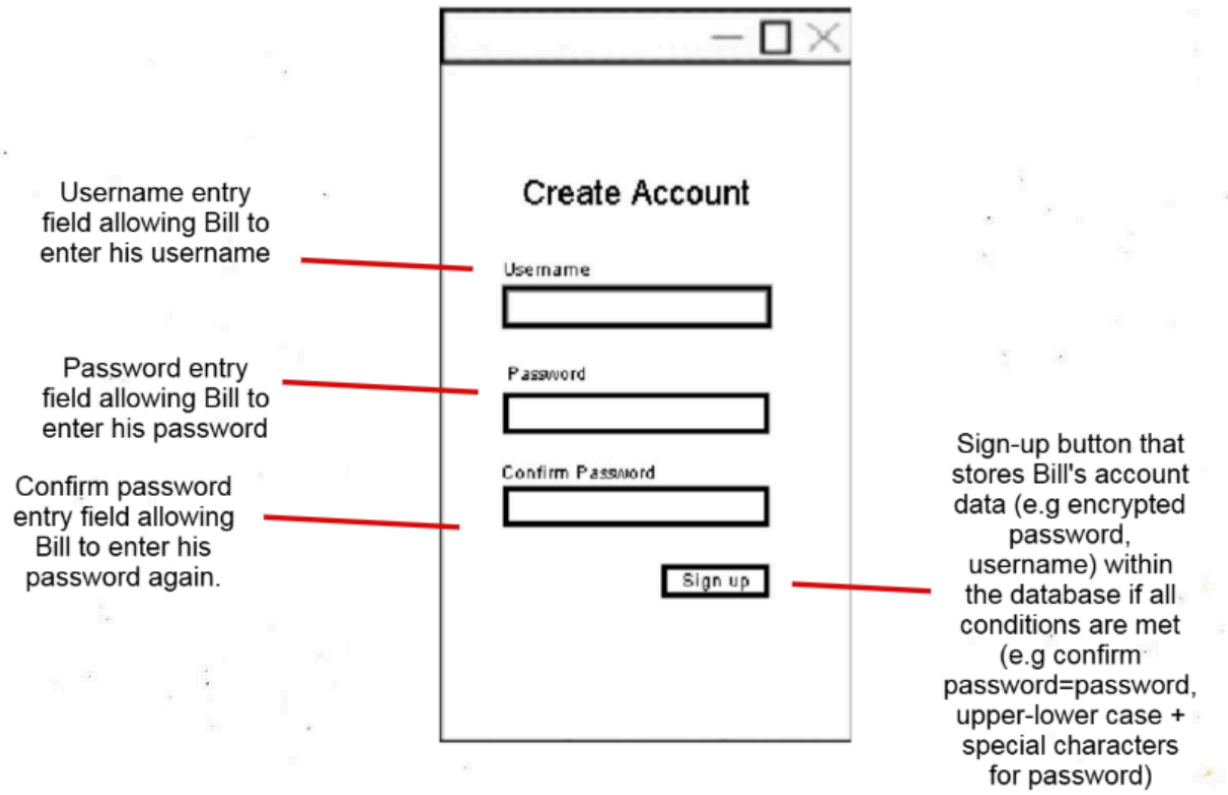


Figure 3: User account creation tab

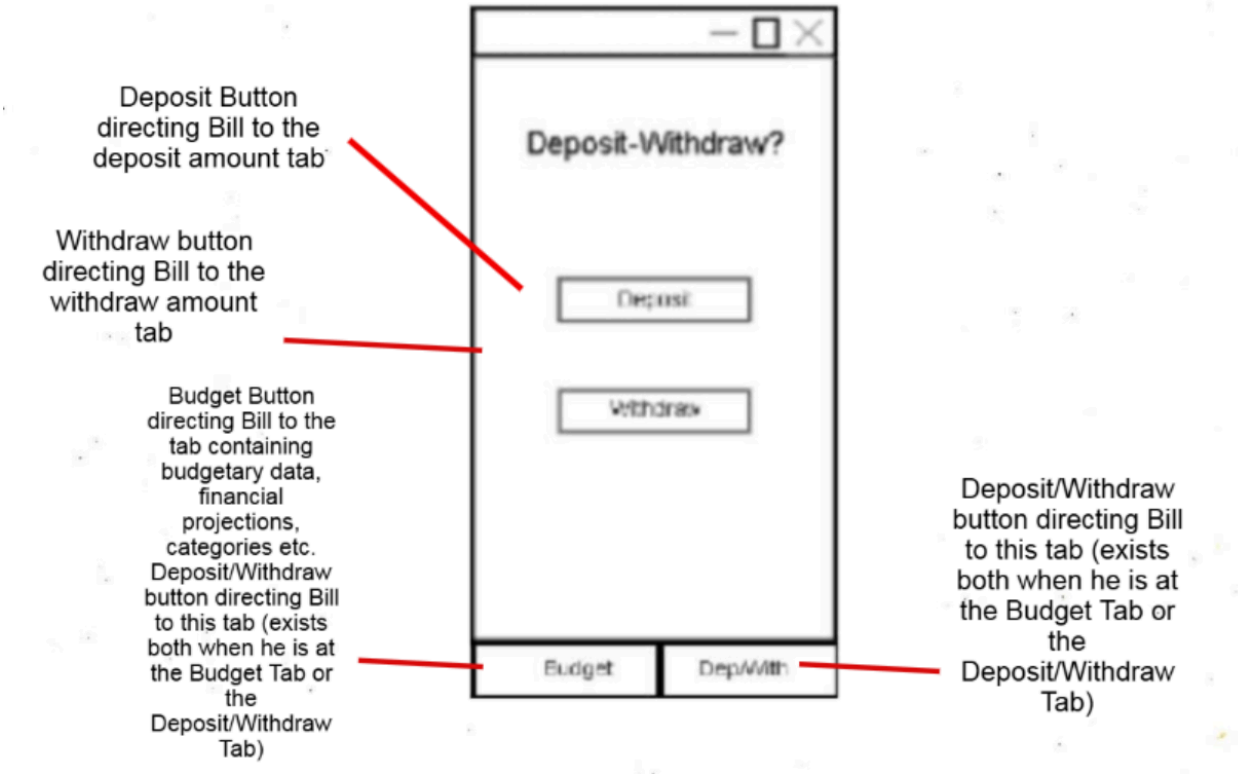
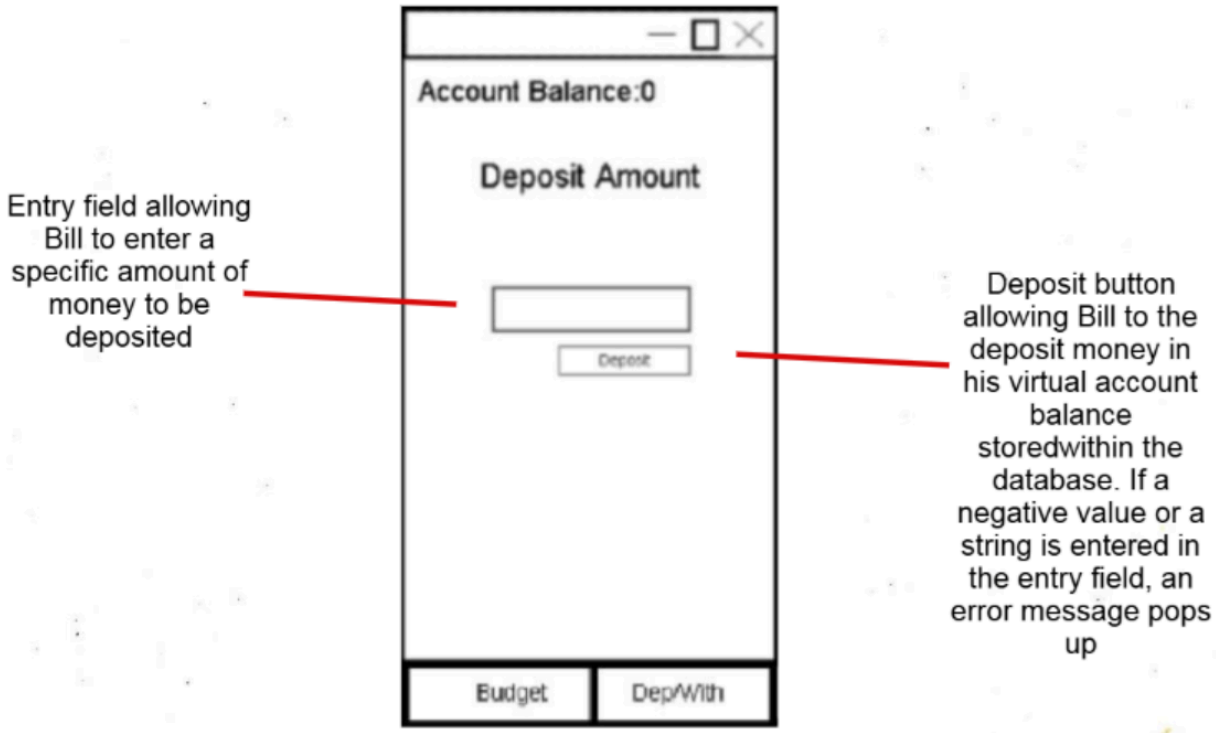


Figure 4: Deposit-Withdraw tab



SFigure 5: Deposit tab

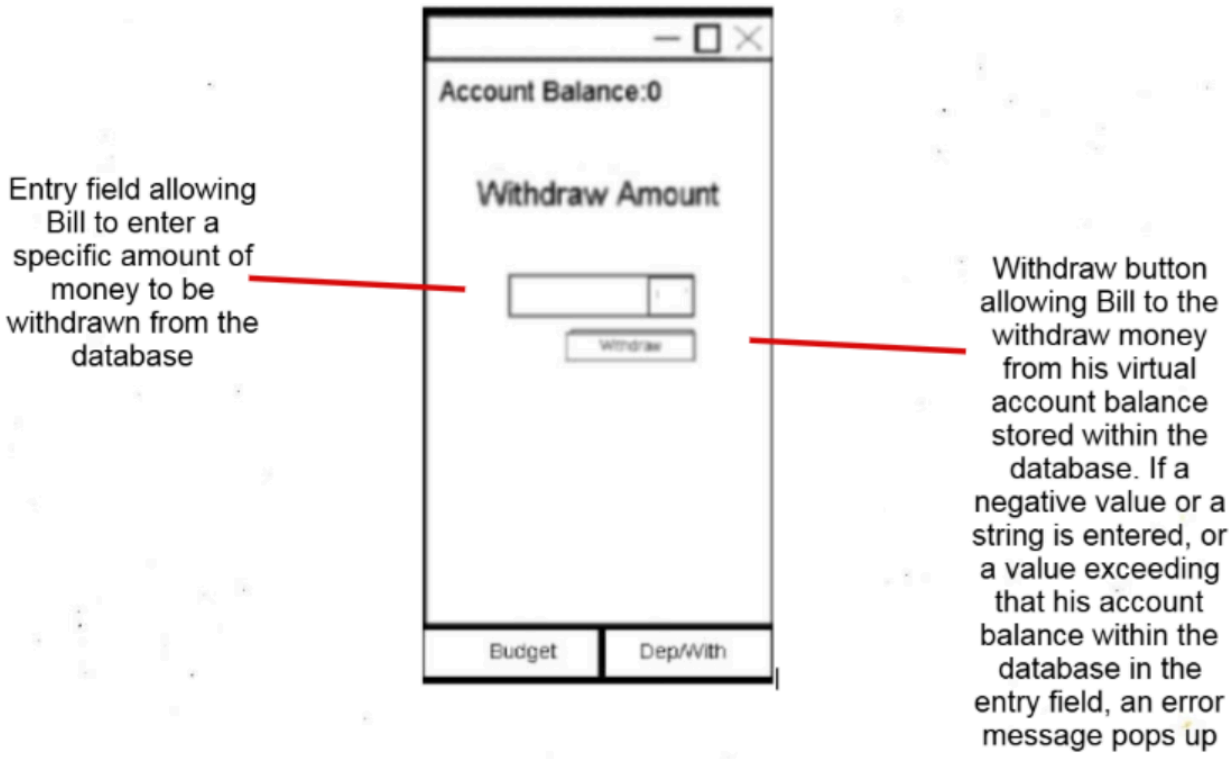


Figure 6: Withdraw amount tab

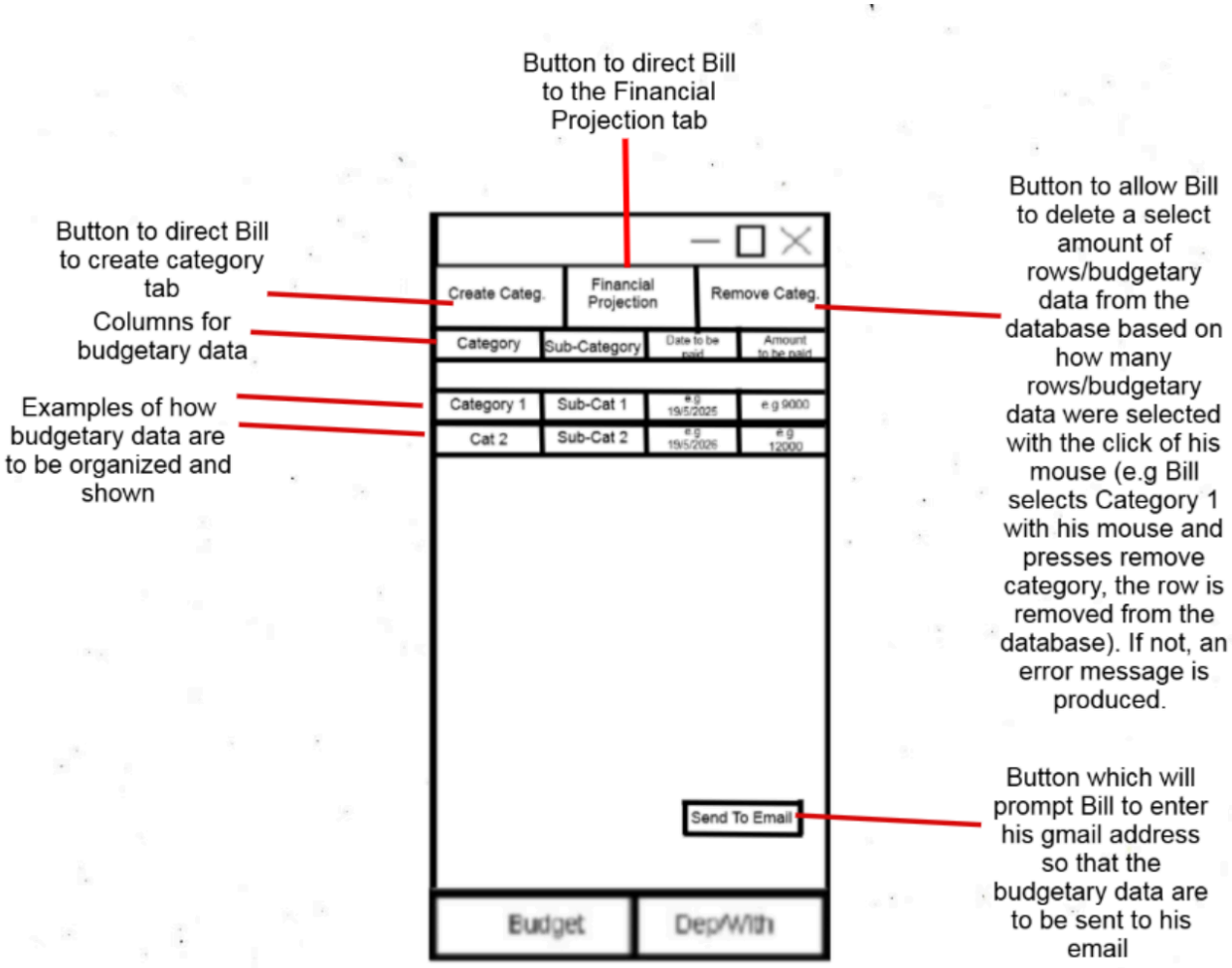


Figure 7: Main Budget Tab

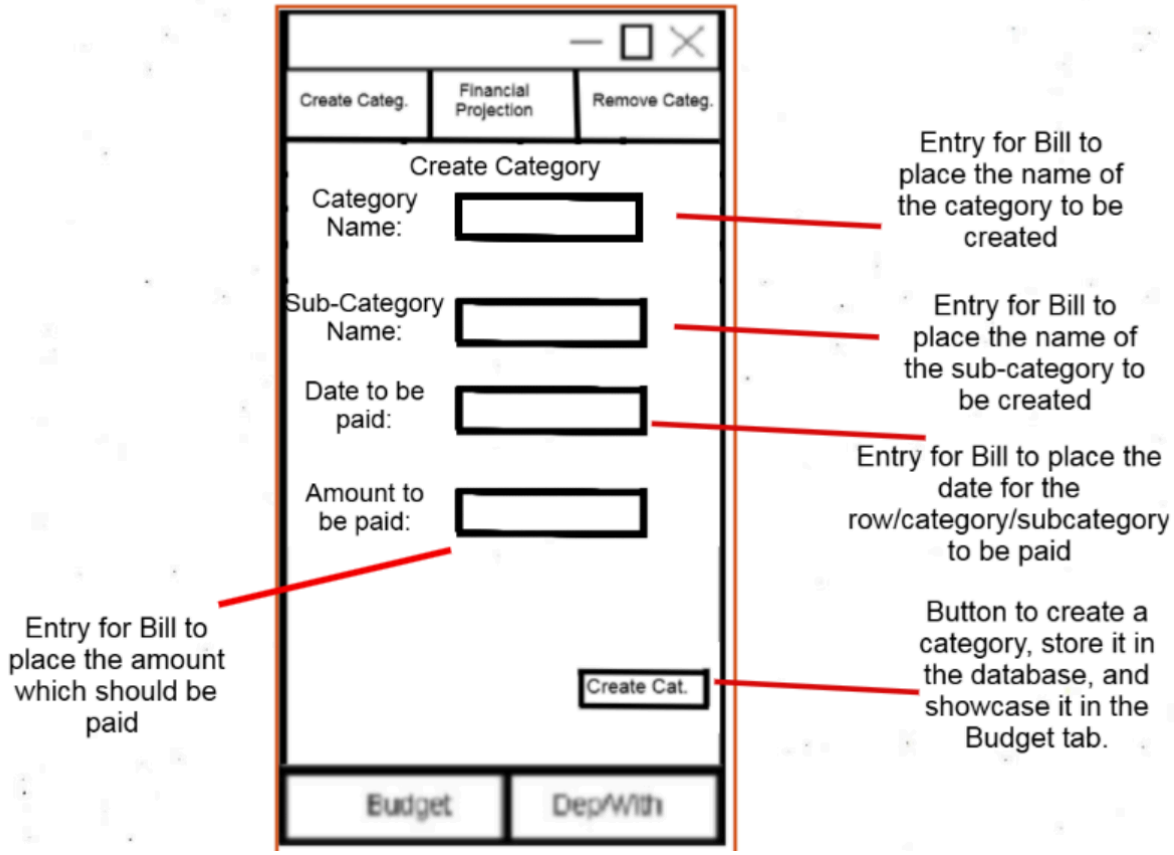


Figure 8: Create Category Tab

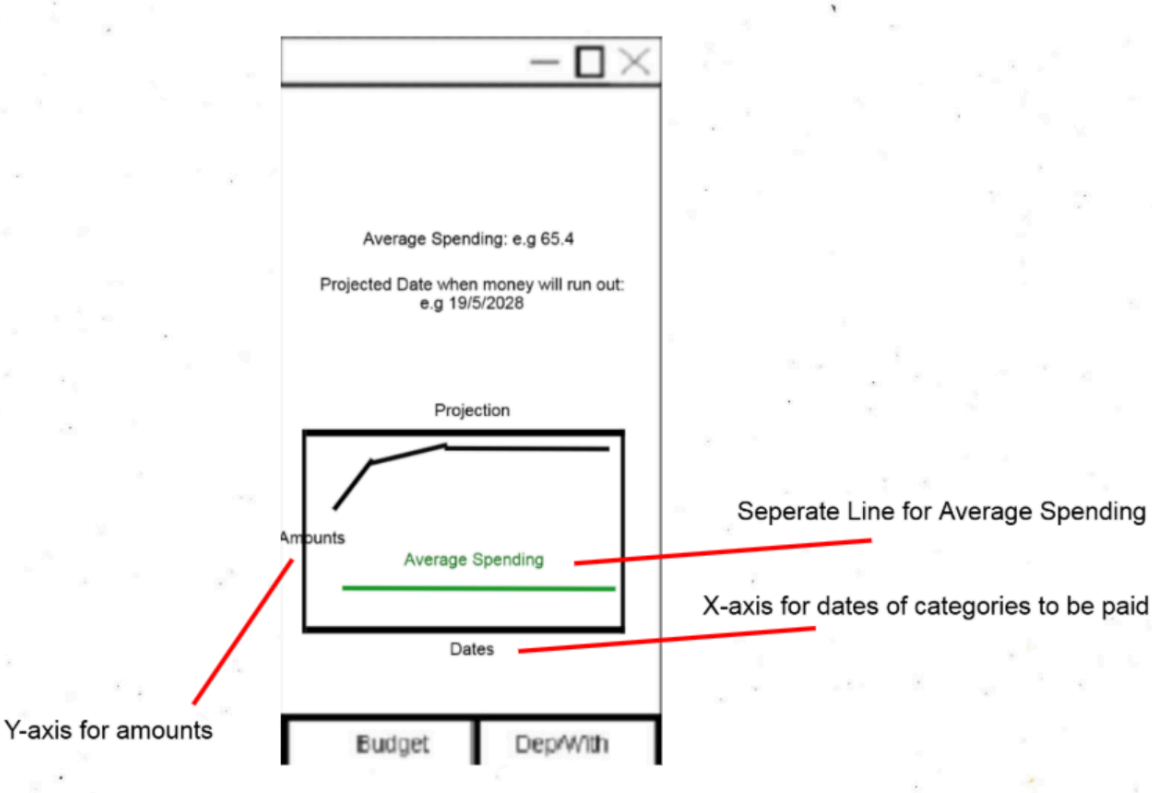


Figure 9: Financial Projection Tab

UML Diagrams:

The following are UML diagrams showing overall relationships within the code and architecture between functions and classes.

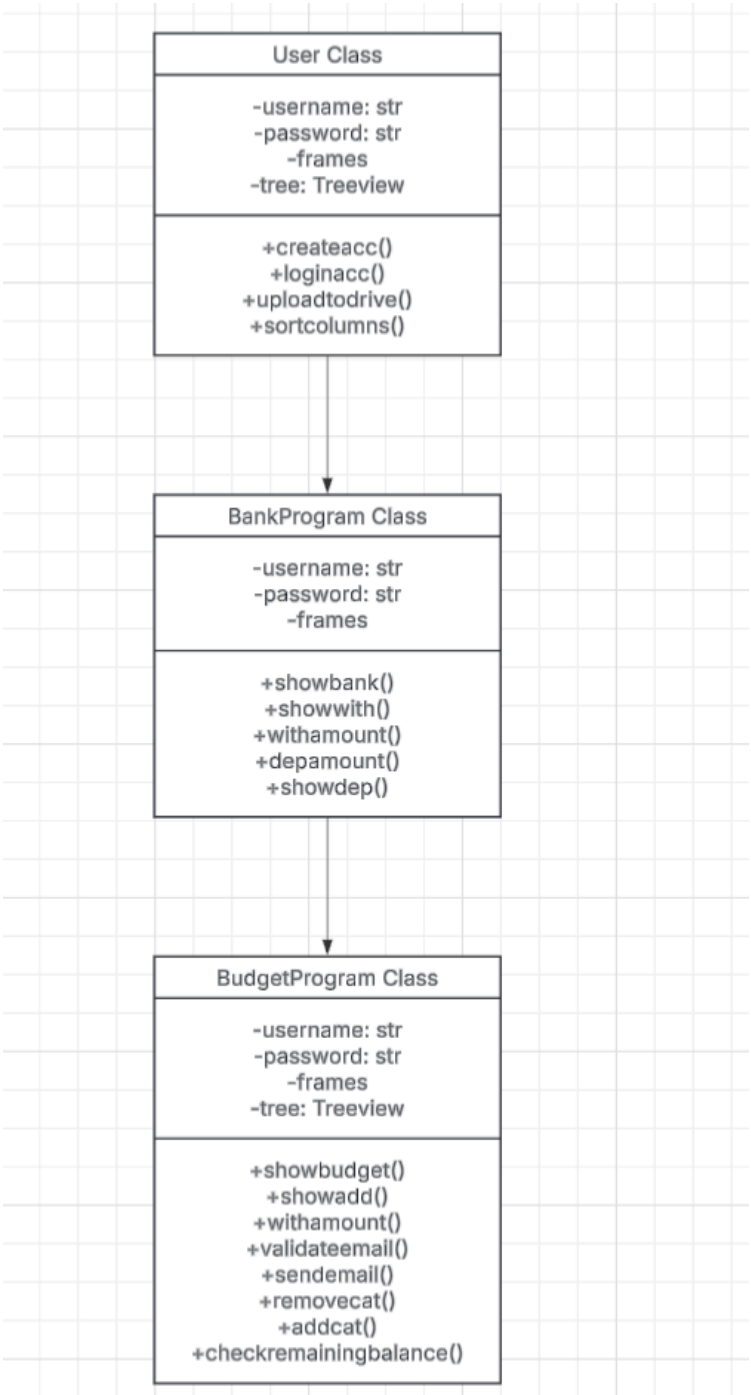


Figure 10: UML Diagram of Classes with functions made using LucidChart

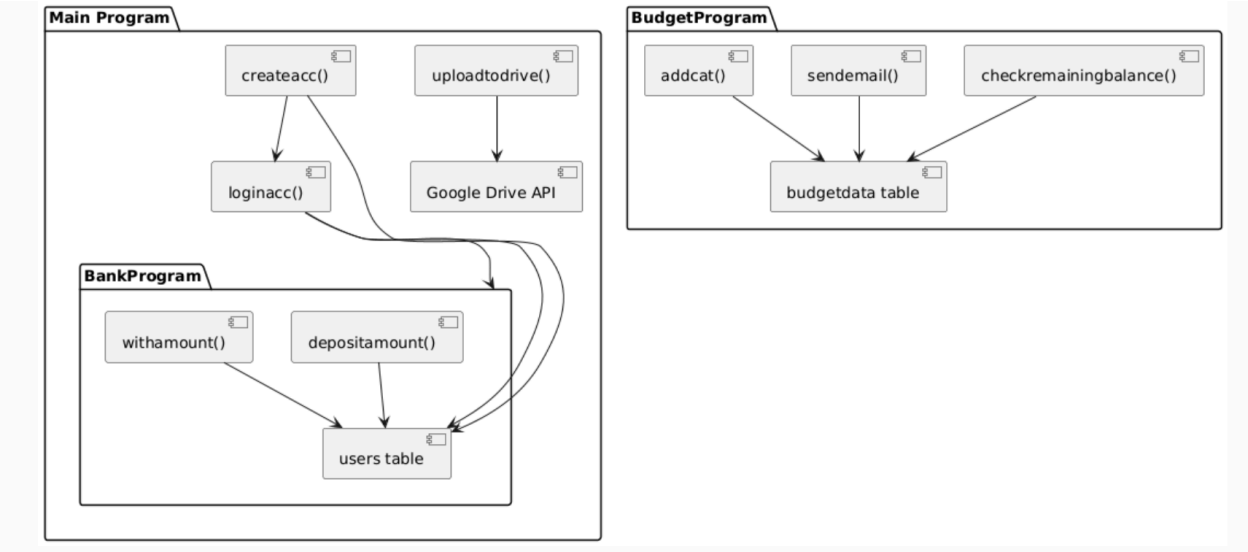


Figure 11: Simple UML Diagram showcasing some relationships between functions made using PlantUML

Flowcharts:

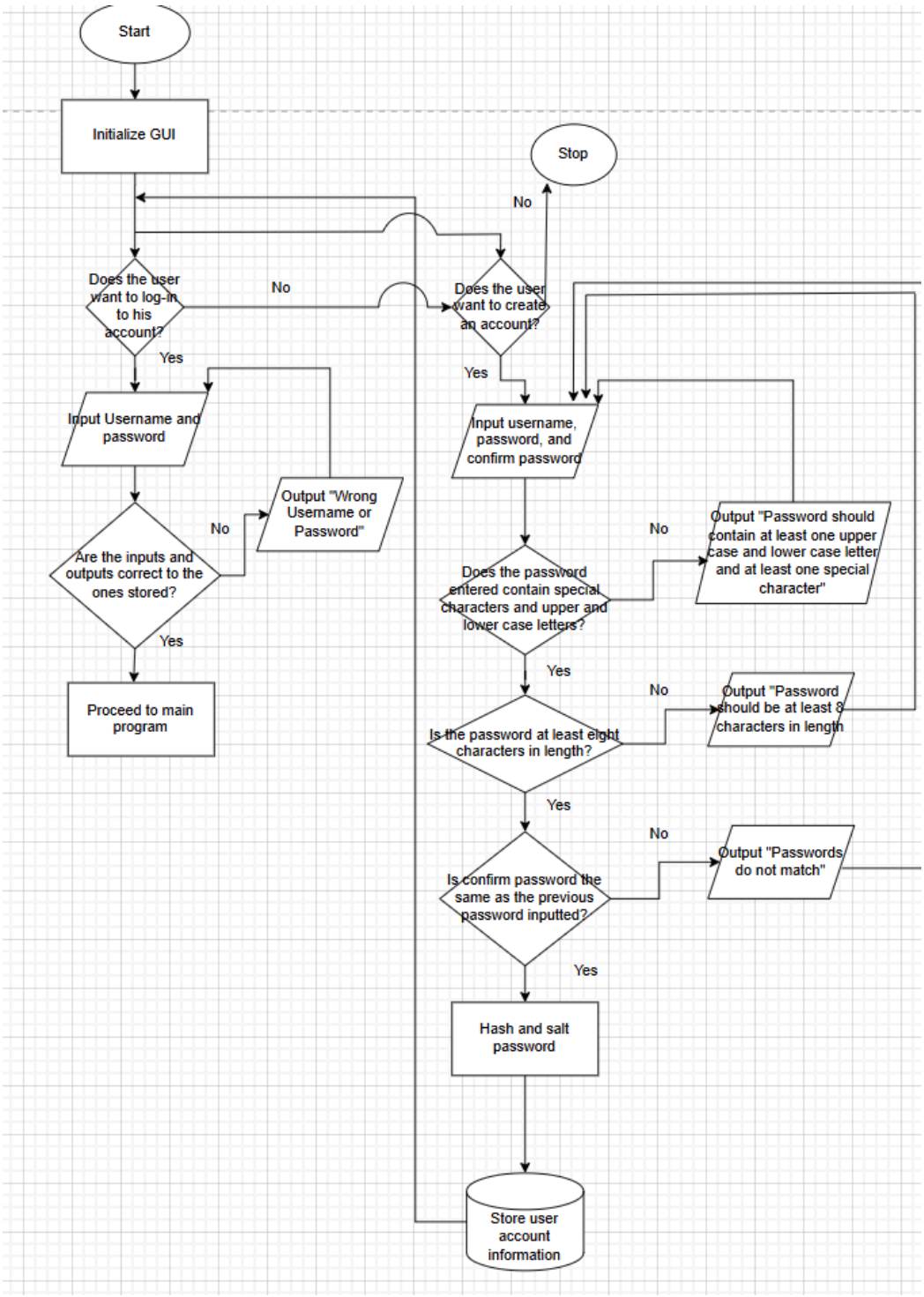


Figure 10: Systems flowchart showcasing when the user is prompted to input whether to create a new account or log in to an existing one up until the account is either created or the user logs into his account.

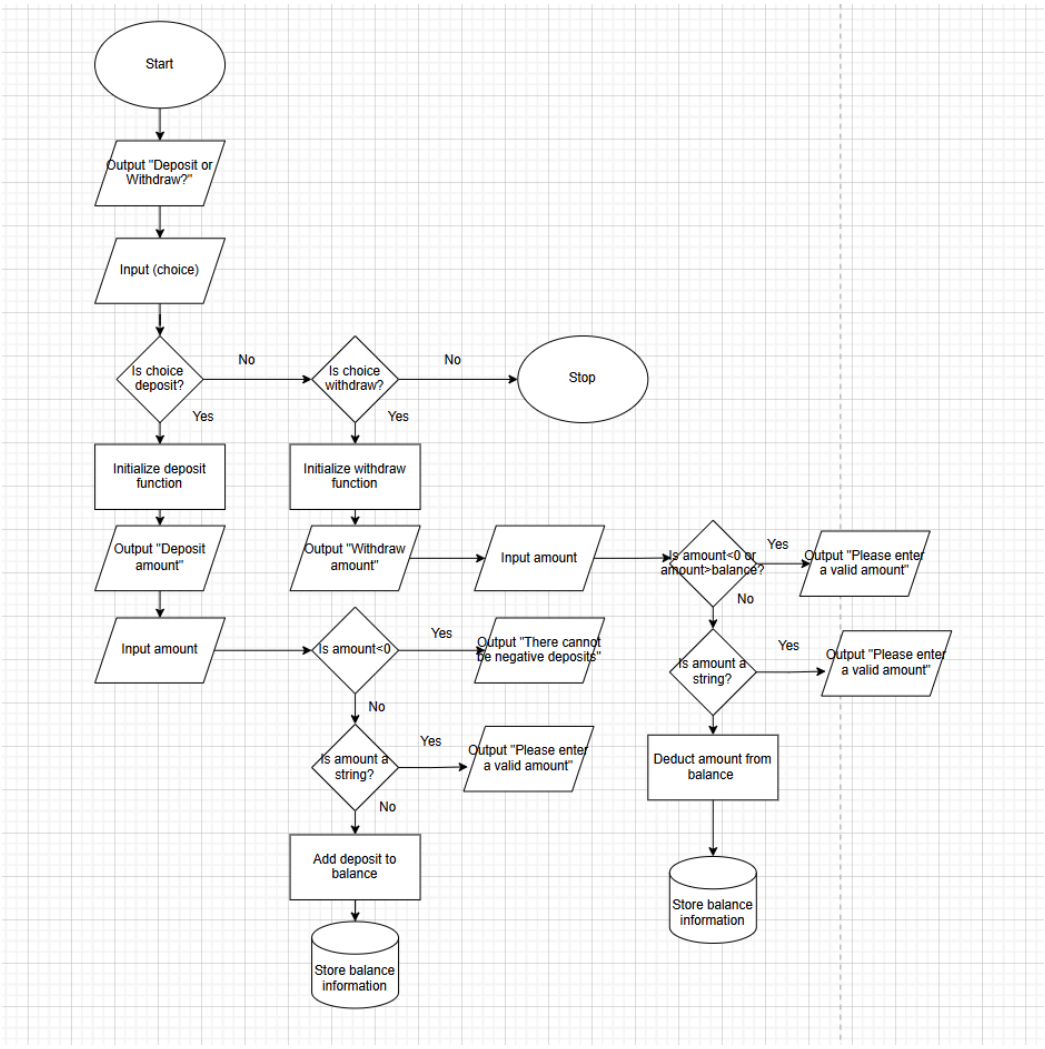


Figure 11: Systems flowchart showcasing every possible option and outcome of the deposit and withdrawal section of the program.

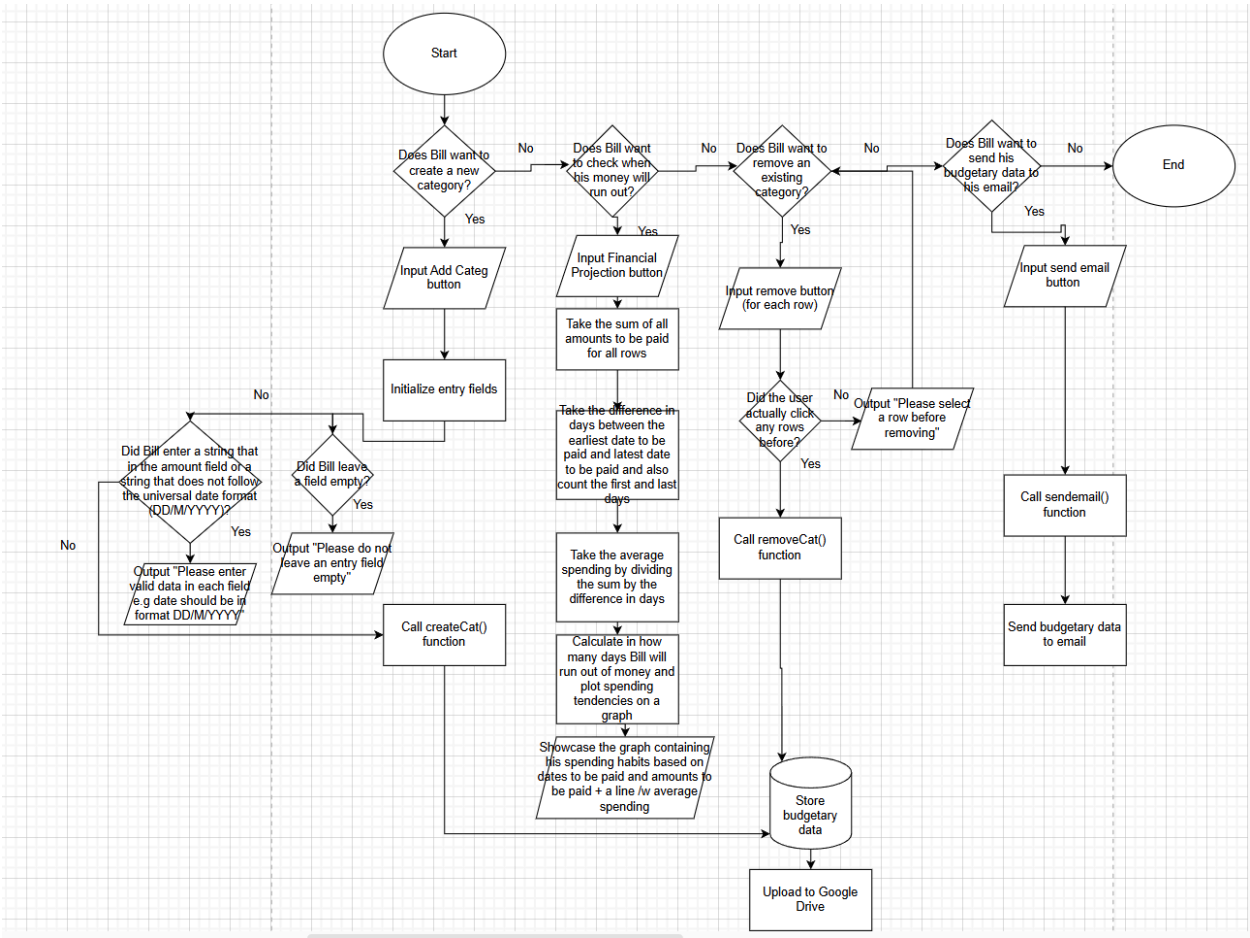


Figure 12: System flowchart regarding the budget section. Showcases each option that can be chosen as well as the underlying processes of each option. Every choice the user makes is also stored in the database.

Pseudocode:

```
1
2  method sortcolumns()
3      Define rowscollection as an empty list
4
5      For each row in the rowscollection:
6          Fetch values from the specified column (col) and store it along with the row identifier
7          rowscollection.addItem(value,rowidentifier)
8      |
9      #Since python has built in sort() based on Timsort, bubble or selection sort are not needed
10     If column == "Amount":
11         Sort rowscollection based on numeric value (value) either in reverse or vice versa
12     Else if column == "Date":
13         Sort rowscollection based on date (value), either in reverse or vice versa
14     Else:
15         Sort rowscollection depending on subcategory or category columns either in reverse or vice
16 end method sortcolumns
```

Figure 13: Preliminary pseudocode regarding sortcolumn()

```
method validatemail(email)
    COUNT=0
    for c in email:
        if c=='@' Then
            COUNT++
    if COUNT!=1 or email ends with(@gmail.com) Then
        return False
    else
        return True
end method validatemail


method sendemail():
    FLAG = TRUE
    cancelbutton=False
    email=input("Enter an email: ")

    loop while (!email or !validatemail(email) or cancelbutton!=True) AND FLAG=TRUE:
        if !email:
            break

        if not validate_email(email):
            output("Invalid email address, please enter a valid email.")
            email=input("Enter an email: ")

        if cancelbutton==True:
            FLAG=False
        else if validateemail(email):
            FLAG=False

    if cancel_button == True:
        return

    send email to email
end method sendemail

```

Figure 14: Preliminary pseudocode regarding validateemail() and sendemail()

Basic SQLite Database Schema:

Table 1: Table regarding user account creation and login

Name	Data type	Rules	Explanation
‘id’	INTEGER	NOT NULL, UNIQUE	Identifier for Bill
‘username’	TEXT	NOT NULL, UNIQUE	Username of Bill

‘password’	TEXT	NOT NULL	Password of Bill (to be salted and hashed)
‘balance’	REAL	UNIQUE	Contains Bill’s account balance

Table 2: Table with columns containing budgetary information of Bill

Name	**Data type**	**Rules**	**Explanation**
‘id’	INTEGER	NOT NULL, UNIQUE	Identifier for Bill
‘category’	TEXT	NOT NULL	Category of budget data
‘amount’	REAL	NOT NULL	Amount needed to be paid for each budget
‘date’	TEXT	NOT NULL	Date for an amount to be paid
‘subcategory’	TEXT	NOT NULL	Subcategory of budget data

Test Plan:

Key: Normal data, Abnormal data

Success Criteria No.	Test Description	Input data	Expected result
1.	Bill can deposit to his virtual account balance	Bill attempts to deposit a certain amount of money and clicks the "Deposit Amount" button Bill attempts to deposit an amount that is negative	That certain amount of money are added to his account balance in the database An appropriate error message should be produced
1.	Bill can withdraw from his virtual account balance	Bill attempts to deposit a certain amount of money and clicks the "Withdraw Amount" button Bill attempts to	That certain amount of money is deducted from his account balance in the database An appropriate

		withdraw an amount that is negative or the amount to be withdrawn exceeds his account balance	error message should be produced
2.	Bill's previously inputted budgetary data (e.g amounts and dates to be paid), prior to shutdown, are stored in the database and shown on the screen	Bill previously inputted and stored in the database: 85 as an amount for a category whose name is "Food", with subcategory "Banana" and then he set a date to be paid	Show the date to be paid, the amount previously entered and the name of the category and subcategory
2.	Bill can login his account using the credentials inputted to the database upon account creation	Bill's log-in information inputted within the program's entry fields in the log-in tab match those within the database Bill enters login credentials that do not match those within the	Bill can access the rest of the program An appropriate error message should be produced

		database	
3.	Bill can create a new row of budgetary data which includes category and subcategory name, date and amount to be paid using the createcat() function	Bill inputs strings for category and subcategory fields, a string of appropriate DD/MM/YYYY format for date to be paid, and a float for amount to be paid and clicks the “Create Category” button	A new row of budgetary data is stored in the database
3.	Bill can delete selected rows of budgetary data which includes category and subcategory name, date and amount to be paid using the removecat() function	Bill selects using his mouse all the rows to be deleted and clicks the “Remove category” button	All selected rows of budgetary data are deleted from the database
4.	The password entered by Bill upon account creation is to be hashed and salted within the database	Bill creates an account using the “Create Account” button	The password is stored in the database as a series of random characters, unidentical to the one

			inputted by Bill
5.	A button should be placed alongside his budgetary data prompting Bill to enter his email address once pressed	Bill presses the “Send Email” button	A new pop-up window is created with an entry field where the email address is to be placed in appropriate email format
5.	If Bill enters a correct gmail address, there should be communication with gmail servers and the budgetary data are to be shown in his email	Bill enters the email correctly and presses “send” again	His budgetary data are shown in a new email from an email address created by me
6.	Once Bill shuts down the application, the database is saved to a google drive folder created by me using an API	Bill closes the application	A new .db file becomes part of the database
7.	Rows are to be sorted either alphabetically, numerically, or by date depending on the column header clicked by Bill	Bill presses the “Amounts” column	Rows of budgetary data are sorted numerically by amount

8.	Special characters "!@#$%^&*()-+?_=,<>/", at least one instance of a capital letter, and a password with more than 8 characters are to be inputted by the user for password creation to allow Bill to access the program	Bill inputs a password with special characters, at least one capital letter, and more than 8 characters in size If at least one is missing	Proceed to the main program Produce an error message
9.	Based on the total amounts of each row of budgetary data and the difference between the most distant and least distance date for a category to be paid, a financial projection for when Bill's money will run out based on average spending should be produced	Bill presses the "Financial Projection" button	Output average daily spending and projected date when money will run out
10.	Based on the amount to be spent and the date to be spent corresponding to each row of budgetary data, a graph should be produced that passes through each value plotted against the x and y-axis	Bill presses the "Financial Projection" button	Output a graph showcasing when the spending trends of Bill based on the amounts to be paid and dates for each budgetary data

			to be paid. A separate graph should also be shown including average salary.
11.	A connection to a database file should be established when Bill starts the application allowing the fetching and manipulation of data depending on changes made within the application	Bill starts the application and creates an account	The program knows the location of the database within the directory and makes updates depending on Bill's choices
12.	If Bill attempts to enter empty fields an appropriate error message should be produced	Bill leaves an entry field empty	An appropriate error message should be produced e.g "Please enter all fields"
12.	Fields should only accept the data type for which they are originally assigned	Bill attempts to enter a string as an amount to be paid or an invalid email syntax	An appropriate error message should be produced e.g "Please enter an an appropriate format in the field"